

NOTE:

The 1st transaction is called the one shown on the left in the screenshots, and the 2nd is called the one on the right.

- Create a table called employee with columns id serial, name varchar, status varchar.
- Replicate the example given in the lecture with the code:

– 1st scenario

Two separate transactions are started. The insert occurred in the 1st transaction that is not visible in the 2nd transaction

The left screenshot (Script-8) shows the following SQL code:

```
CREATE TABLE public.employee (  
-- first transaction  
begin;  
select txid_current();  
insert into public.employee ("name", status)  
values ('Alice', 'Not fired');  
select *, xmin, xmax from public.employee e;  
commit;
```

The right screenshot (Script-9) shows the following SQL code:

```
-- first transaction  
begin;  
select *, xmin, xmax from public.employee;  
commit;  
-- second transaction  
begin;  
select txid_current();  
delete from public.employee  
where id = 1;  
select *, xmin, xmax  
from public.employee e;
```

Both screenshots show a table with columns: id, name, status, xmin, xmax. The left window shows a single row with id 1, name Alice, status Not fired, xmin 980, and xmax 0. The right window shows an empty table.

After the committing 1st transaction, the inserted row is appeared in the second transaction - **non-repeatable read anomaly** happened in the scope of 2nd transaction - so, we used **'Read committed'** isolation level

Script-8

```
CREATE TABLE public.employee (  
-- first transaction  
begin;  
select txid_current();  
insert into public.employee ("name", status)  
values ('Alice', 'Not fired');  
select *, xmin, xmax from public.employee e;  
commit;
```

Script-9

```
-- first transaction  
begin;  
select *, xmin, xmax from public.employee;  
commit;  
  
-- second transaction  
begin;  
select txid_current();  
delete from public.employee  
where id = 1;  
select *, xmin, xmax  
from public.employee e;
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.002s
Start time	Wed Jul 08 22:18:41 EEST 2026
Finish time	Wed Jul 08 22:18:41 EEST 2026
Query	commit

employee 1

id	name	status	xmin	xmax
1	Alice	Not fired	980	0

– 2nd scenario

The row was deleted in the scope of uncommitted 2nd transaction, so it is still visible on the SELECT command of 1st transaction and disappeared after the 2nd transaction commit

Script-8

```
-- first transaction  
begin;  
select txid_current();  
insert into public.employee ("name", status)  
values ('Alice', 'Not fired');  
select *, xmin, xmax from public.employee e;  
commit;  
  
-- second transaction  
begin;  
select *, xmin, xmax  
from public.employee;  
commit;
```

Script-9

```
begin;  
select *, xmin, xmax from public.employee;  
commit;  
  
-- second transaction  
begin;  
select txid_current();  
delete from public.employee  
where id = 1;  
select *, xmin, xmax  
from public.employee e;  
commit;
```

employee 1

id	name	status	xmin	xmax
1	Alice	Not fired	980	981

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.002s
Start time	Wed Jul 08 22:27:26 EEST 2026
Finish time	Wed Jul 08 22:27:26 EEST 2026
Query	delete from public.employee where id = 1

***<postgres> Script-8 X**

```
-- first transaction
begin;

select txid_current();

insert into public.employee ("name", status)
values ('Alice', 'Not fired');

select *, xmin, xmax from public.employee e;

commit;

-- second transaction
begin;

select *, xmin, xmax
from public.employee;

commit;
```

employee 1 X

select *, xmin, xmax from public.employee

id	name	status	xmin	xmax
123	Alice	Not fired		

***<postgres> Script-9 X**

```
begin;

select *, xmin, xmax from public.employee e;

commit;

-- second transaction
begin;

select txid_current();

delete from public.employee
where id = 1;

select *, xmin, xmax
from public.employee e;

commit;
```

Statistics 1 X

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Wed Jul 08 22:29:16 EEST 2026
Finish time	Wed Jul 08 22:29:16 EEST 2026
Query	commit

– 3rd scenario

The 2nd transaction updates the table data change is not reflected in the scope of 1st transaction before commit but displayed after the commit

```
select *, xmin, xmax
from public.employee;

commit;

insert into public.employee ("name", status)
values ('Alice', 'Not fired');

-- third transaction
begin;

select *, xmin, xmax
from public.employee e;

commit;
```

employee 1 X

select *, xmin, xmax from public.employee

id	name	status	xmin	xmax
2	Alice	Not fired	982	983

```
delete from public.employee
where id = 1;

select *, xmin, xmax
from public.employee e;

commit;

-- third transaction
begin;

select txid_current();

update public.employee
set status = 'Fired'
where id = 2;

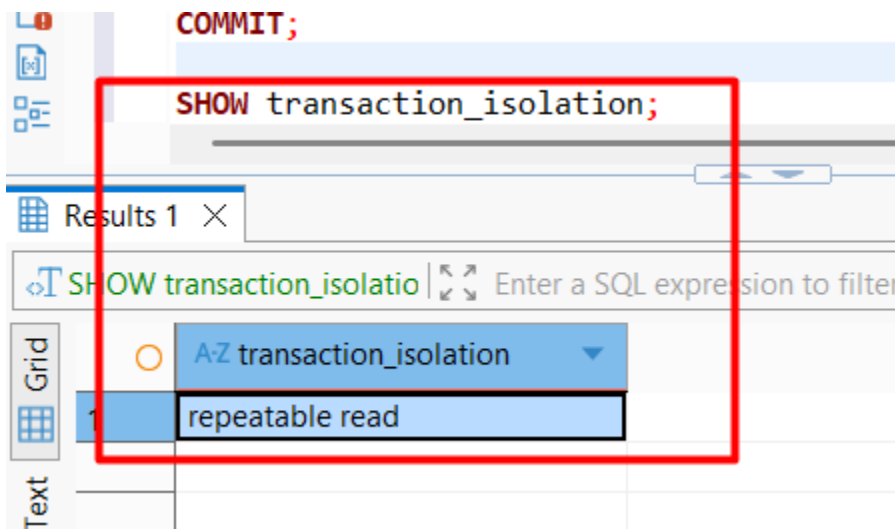
select *, xmin, xmax
from public.employee e;
```

employee 1 X

select *, xmin, xmax from public.employee

id	name	status	xmin	xmax
1	2	Alice	Fired	983
2				0

–Run the command set transaction isolation level repeatable read.

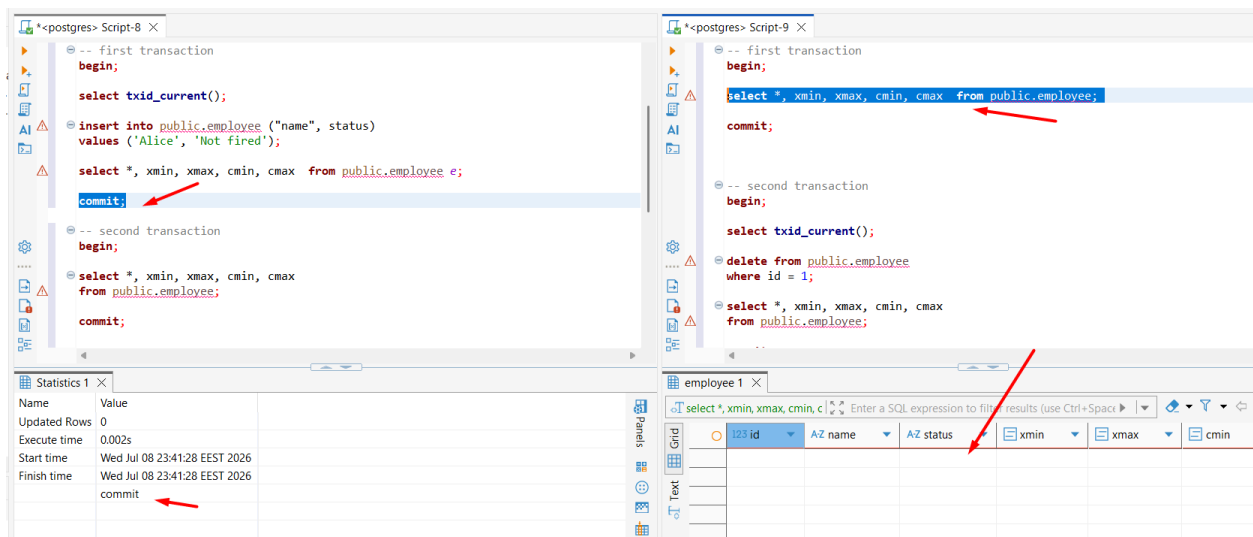


–Check your current isolation level in each session with show transaction isolation level.

– 1st scenario

Two separate transactions are started. The insert occurred in the 1st transaction that is not visible in the 2nd transaction

After the committing 1st transaction, the inserted row is **NOT** appeared in the second transaction - **non-repeatable read anomaly** is **gone** in the scope of 2nd transaction - so, we used '**Repeatable read**' isolation level



– 2nd scenario

The row was deleted in the scope of uncommitted 2nd transaction, so it is still visible on the SELECT command of 1st transaction and **NOT** disappeared after the 2nd transaction commit

(1st is still uncommitted)

Script-8

```
values ('Alice', 'Not fired');
select *, xmin, xmax, cmin, cmax from public.employee e;
commit;
-- second transaction
begin;
select *, xmin, xmax, cmin, cmax
from public.employee;
commit;
insert into public.employee ("name", status)
values ('Bob', 'Not fired');
```

Script-9

```
-- second transaction
begin;
select txid_current();
delete from public.employee
where id = 1;
select *, xmin, xmax, cmin, cmax
from public.employee;
commit;
-- third transaction
begin;
select txid_current();
```

id	name	status	xmin	xmax	cmin
1	Alice	Not fired	1011	1012	0

Name	Value
Updated Rows	0
Execute time	0.002s
Start time	Wed Jul 08 23:56:11 EEST 2026
Finish time	Wed Jul 08 23:56:11 EEST 2026
commit	

– 3rd scenario

The 2nd transaction updates the table data change is not reflected in the scope of 1st transaction before and **after** commit (1st is still uncommitted)

Script-8

```
select txid_current();
insert into public.employee ("name", status)
values ('Alice', 'Not fired');
select *, xmin, xmax, cmin, cmax from public.employee e;
commit;
-- second transaction
begin;
select *, xmin, xmax, cmin, cmax
from public.employee;
commit;
insert into public.employee ("name", status)
values ('Bob', 'Not fired');
```

Script-9

```
select *, xmin, xmax, cmin, cmax
from public.employee;
-- third transaction
begin;
select txid_current();
update public.employee
set status = 'Fired'
where id = 2;
select *, xmin, xmax
from public.employee e;
commit;
```

id	name	status	xmin	xmax	cmin
1	Alice	Not fired	1013	1014	0

Name	Value
Updated Rows	0
Execute time	0.002s
Start time	Thu Jul 09 00:15:40 EEST 2026
Finish time	Thu Jul 09 00:15:40 EEST 2026
Query	commit

–Recreate employee table and redo the second task but modify the code so that select statements would now include cmin and cmax system columns. What changed?

Nothing changed when the tasks flow repeated. Because we had one query per transaction. To see these columns change, a few inserts within 1 transaction were performed.

employee 1		employee 2	employee 2 (2) ×	Statistics 2		
Insert into public.employee		Data filter is not supported				
	AZ name	AZ status	xmin	xmax	cmin	cmax
2	Alice	Fired	1015	0	0	0
3	Bob	Not fired	1015	0	1	1
4	Carol	Not fired	1015	0	2	2